

Why P^{SAT} Contains Both NP and coNP

A complete proof using the definition of oracle computation

Theorem.

$$NP \subseteq P^{SAT} \quad \text{and} \quad coNP \subseteq P^{SAT}.$$

In words: a deterministic polynomial-time Turing machine equipped with an oracle for SAT can decide every language in NP and every language in coNP.

Background and plan

Recall that SAT is the language of satisfiable Boolean formulas. By the Cook-Levin theorem, SAT is NP-complete under polynomial-time many-one reductions. Concretely, this means that for every language $L \in NP$, there exists a polynomial-time computable function f such that for every input x ,

$$x \in L \iff f(x) \in SAT.$$

An oracle machine for SAT may, in one step, ask whether a string written on its oracle tape belongs to SAT. The strategy is therefore straightforward:

- For a language in NP, reduce the input to a Boolean formula and ask the oracle whether that formula is satisfiable.
- For a language in coNP, reduce membership in its complement to SAT, ask the same oracle question, and reverse the answer.

We now carry this out carefully.

Part I: Showing $NP \subseteq P^{SAT}$

Let L be any language in NP. Since SAT is NP-complete, there exists a polynomial-time computable reduction

$$f : \Sigma^* \rightarrow \Sigma^*$$

such that for all strings x ,

$$x \in L \iff f(x) \in SAT.$$

We construct a deterministic oracle Turing machine M^{SAT} deciding L as follows.

1. On input x , compute $f(x)$ deterministically.
2. Write $f(x)$ on the oracle tape.
3. Enter the QUERY state.
4. If the oracle answers YES, accept; if it answers NO, reject.

This machine is deterministic except for the oracle access, and it runs in polynomial time because:

- computing $f(x)$ takes polynomial time, and
- the oracle query itself counts as a single step.

To verify correctness, fix any input x .

$$\begin{aligned} M^{SAT} \text{ accepts } x &\iff \text{the oracle answers YES on } f(x) \\ &\iff f(x) \in SAT \\ &\iff x \in L. \end{aligned}$$

Thus M^{SAT} decides L in deterministic polynomial time with access to a SAT oracle. Since L was arbitrary, we conclude that

$$NP \subseteq P^{SAT}.$$

Part II: Showing $\text{coNP} \subseteq \text{P}^{\text{SAT}}$

Now let L be any language in coNP . By definition, its complement $\bar{L}$ lies in NP . Applying the result of Cook-Levin to $\bar{L}$, there exists a polynomial-time computable function g such that for every input x ,

$$x \in \bar{L} \iff g(x) \in \text{SAT}.$$

Equivalently,

$$x \in L \iff g(x) \notin \text{SAT}.$$

We again build a deterministic polynomial-time oracle machine N^{SAT} .

1. On input x , compute $g(x)$ deterministically.
2. Write $g(x)$ on the oracle tape.
3. Enter the QUERY state.
4. If the oracle answers YES, reject; if it answers NO, accept.

This machine also runs in polynomial time for the same reason as before: the only substantial work is computing $g(x)$, which is polynomial, and the oracle call is one step.

For correctness, again fix any input x .

$$\begin{aligned} N^{\text{SAT}} \text{ accepts } x &\iff \text{the oracle answers NO on } g(x) \\ &\iff g(x) \notin \text{SAT} \\ &\iff x \in L. \end{aligned}$$

Therefore N^{SAT} decides L in polynomial time with access to a SAT oracle. Since L was arbitrary in coNP , we obtain

$$\text{coNP} \subseteq \text{P}^{\text{SAT}}.$$

A short conceptual summary

The reason this works is that a SAT oracle instantly solves the canonical complete problem for NP. Since every NP language reduces to SAT, a polynomial-time machine can solve any NP problem by first carrying out the reduction and then asking the oracle one question. For a coNP language, the complement is in NP, so the same reduction works, except that the machine reverses the oracle's answer.

Conclusion

We have proved both inclusions:

$$\text{NP} \subseteq \text{P}^{\text{SAT}} \quad \text{and} \quad \text{coNP} \subseteq \text{P}^{\text{SAT}}.$$

Hence P with a SAT oracle is powerful enough to decide every language in both NP and coNP .